

Hybrid Table Detection and Analysis System Design Document

1. System Overview

The system is designed to detect, extract, and analyze tables from images using a combination of AWS Textract and YOLO object detection. It compares the detected tables with a reference unsanitized (unsan) text file to match and validate table contents.

1.1 Key Features

- Table detection using YOLO neural network model
- Text extraction using AWS Textract
- Table matching using BLEU score comparison
- Support for batch processing of multiple images
- Generation of detailed analysis reports

2. Architecture

2.1 Core Components

1. **TextractProcessor**
 - Handles all AWS Textract interactions
 - Extracts text and table information from images
 - Processes both full images and cropped table regions
2. **UnsanProcessor**
 - Processes unsanitized text files
 - Extracts reference table content and page numbers
 - Maintains mapping between table content and page numbers
3. **Table Detection System**
 - Uses YOLO model for table detection
 - Handles image preprocessing and bounding box extraction
 - Supports both single image and batch processing

2.2 Data Flow

1. Input Processing:
 - Image files are loaded and preprocessed
 - Unsanitized text file is parsed and indexed
 - Configuration settings are loaded
2. Table Detection:
 - YOLO model detects table regions
 - Bounding boxes are extracted and normalized
 - Tables are cropped from original image
3. Text Extraction:

- Textract processes full image and cropped regions
 - Table structure and content are extracted
 - Results are normalized and stored
4. Table Matching:
- BLEU scores are calculated between detected and reference tables
 - Best matches are identified and stored
 - Unmatched tables are tracked separately

3. Component Details

3.1 TextractProcessor Class

Responsibilities: - Initialize AWS Textract client - Process images and extract text - Handle table structure extraction - Manage AWS API interactions

Key Methods: - `get_text_from_cropped_image()` - `get_tables_from_image()` - `extract_tables()` - `extract_text()`

3.2 UnsanProcessor Class

Responsibilities: - Parse unsanitized text files - Extract table content and metadata - Filter tables by page number - Maintain data structure for comparison

Key Methods: - `process_unsan_file()` - `__init__()` with optional page filtering

3.3 Utility Functions

Table Matching: - `get_bleu_mapping()` - `get_matched_unmatched_tables()` - `draw_bounding_boxes()`

YOLO Processing: - `detect_and_crop_tables()` - `process_single_image()`

4. Data Structures

4.1 Table Representation

```
{
    "TableId": str,
    "Text": str,
    "Left": float,
    "Top": float,
    "Width": float,
    "Height": float,
    "PageNo": int
}
```

4.2 Mapping Structure

```
{
    "table_id": str,
    "image_id": str,
```

```
"image_text": str,  
"unsan_text": str,  
"bleu_score": float,  
"page_no": int,  
"line_no": int  
}
```

5. Configuration

5.1 Required Configuration

```
{  
    "region_name": str,  
    "aws_access_key_id": str,  
    "aws_secret_access_key": str  
}
```

5.2 Directory Structure

```
source_dir/  
├── images/  
├── unsan.txt  
├── output/  
└── dump/
```

6. Error Handling

The system implements error handling for: - Image processing failures - AWS Textract API errors - File I/O operations - Invalid configuration - Batch processing errors

7. Technologies and Methods

7.1 Core Technologies

YOLO (You Only Look Once)

- **Purpose:** Used for table detection in images
- **Implementation:** YOLOv10x model
- **Why:**
 - Real-time object detection capabilities
 - High accuracy in detecting structured elements like tables
 - Single-pass detection (faster than region-based approaches)
 - Robust to variations in table styles and formats

AWS Textract

- **Purpose:** Text and table structure extraction
- **Why:**
 - Advanced OCR capabilities
 - Built-in table structure recognition
 - Handles complex document layouts

- High accuracy in text extraction from various fonts and styles
- Automatic handling of rotated text and different languages

BLEU (Bilingual Evaluation Understudy) Score

- **Purpose:** Matching detected tables with reference content
- **How it works:**
 - Compares n-gram sequences between candidate and reference texts
 - Scores range from 0 (no match) to 1 (perfect match)
 - Considers word order and phrase matching
- **Why:**
 - Robust to minor textual variations
 - Good at capturing structural similarities
 - Industry standard for text similarity measurement
 - Better than simple string matching for table content comparison

OpenCV (cv2)

- **Purpose:** Image processing and manipulation
- **Used for:**
 - Image loading and preprocessing
 - Table region cropping
 - Color space conversions
 - Image quality enhancement before OCR
 - Bounding box drawing and visualization

Pandas

- **Purpose:** Data manipulation and analysis
- **Used for:**
 - Table data structuring
 - Results organization
 - CSV file handling
 - Efficient table matching operations
 - Data export and reporting

7.2 Supporting Libraries

NLTK (Natural Language Toolkit)

- **Purpose:** Text processing and BLEU score calculation
- **Why:** Robust implementation of BLEU scoring and text tokenization

Supervision

- **Purpose:** Object detection visualization and processing
- **Why:** Specialized tools for handling YOLO outputs and visualization

Typer

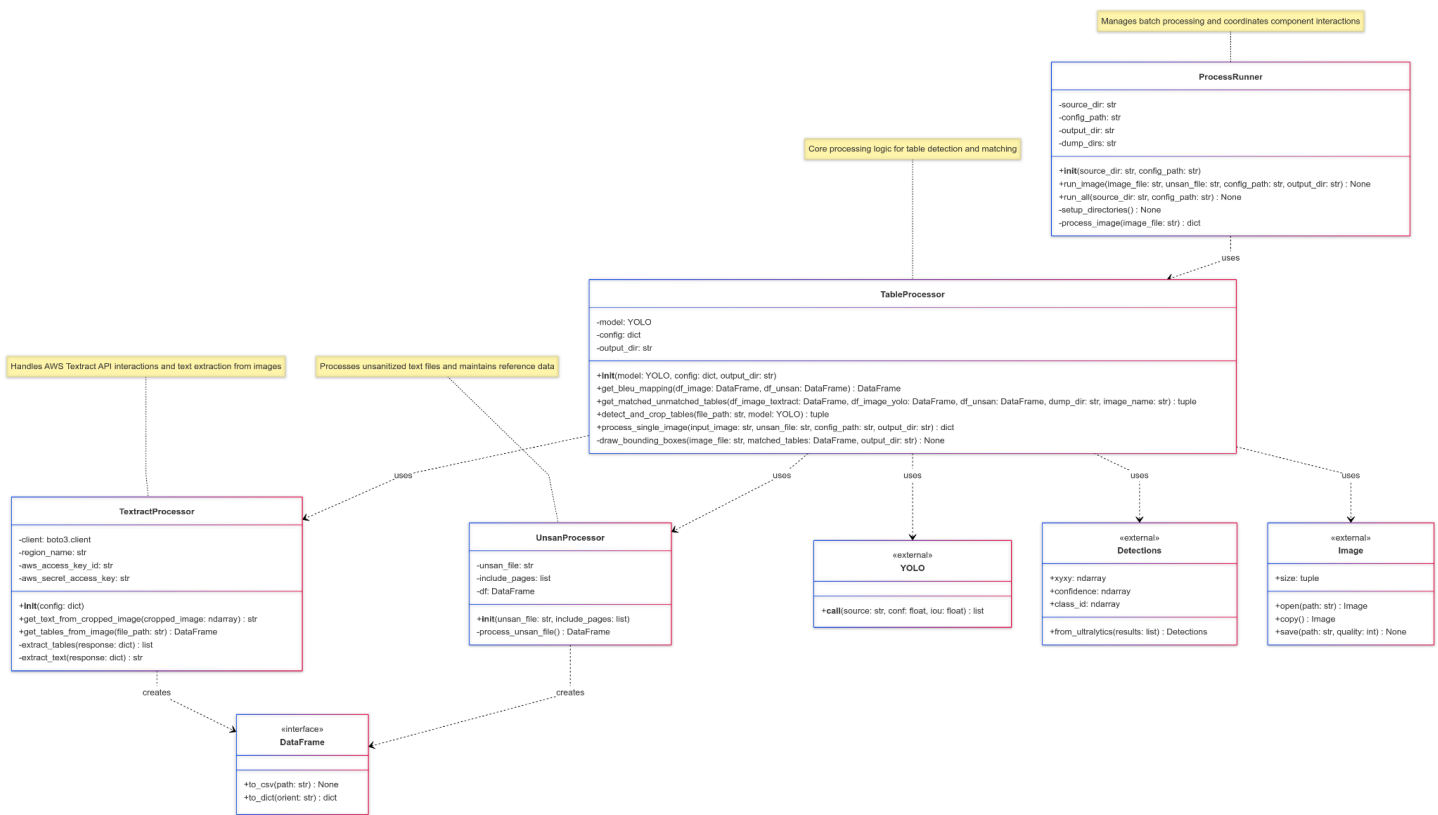
- **Purpose:** Command-line interface creation
- **Why:** Type-safe command-line applications with minimal code

PIL (Python Imaging Library)

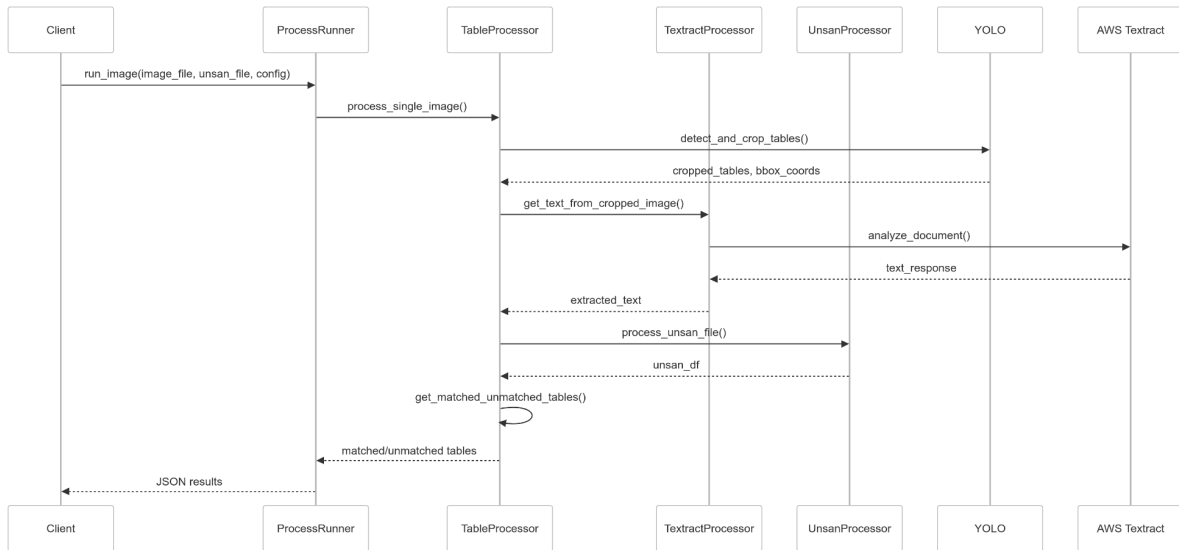
- **Purpose:** Image handling and manipulation
- **Why:** Complementary to OpenCV for certain image operations

8. UML Diagrams

UML Class Diagram



UML Sequence Diagram



9. Output Formats

9.1 JSON Structure

```

{
  "matched": [
    {
      "TableId": "string",
      "Text": "string",
      "Coordinates": {
        "Left": float,
        "Top": float,
        "Width": float,
        "Height": float
      },
      "Confidence": float,
      "BLEU_Score": float
    }
  ],
  "unmatched": [],
  "meta": {}
}

```

9.2 CSV Exports

- Matched tables data
- Unmatched tables data
- Processing statistics
- Error logs

Prerequisites

- Python 3.8 or higher
- AWS account with Textract access
- CUDA-capable GPU (recommended for optimal YOLO performance)

Installation

1. Create and activate a virtual environment (recommended):

```
python -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. Install required packages:

```
pip install -r requirements.txt
```

4. Download NLTK data:

```
import nltk  
nltk.download('punkt')
```

5. Download the YOLO model:

```
# Place your YOLO model file in the /models directory  
mkdir models  
# Copy your yolov10x_best.pt to the models directory
```

Configuration

1. Create a configuration file config.json:

```
{  
  "region_name": "your-aws-region",  
  "aws_access_key_id": "your-access-key",  
  "aws_secret_access_key": "your-secret-key"  
}
```

2. Set up your directory structure:

```
project_root/  
├── images/           # Input images  
├── models/           # YOLO model files  
│   └── yolov10x_best.pt  
├── unsan.txt         # Reference text file  
├── output/           # Output directory  
└── dump/             # Temporary processing files
```

Usage

Single Image Processing

```
python get_unmatched_tables_hybrid.py run-image \  
  --image-file "path/to/image.jpg" \  
  --unsan-file "path/to/unsan.txt" \  
  --config-path "path/to/config.json" \  
  --output-dir "path/to/output"
```

Batch Processing

```
python get_unmatched_tables_hybrid.py run-all \
  --source-dir "path/to/source/directory" \
  --config-path "path/to/config.json"
```

Output Format

The system generates JSON output with the following structure:

```
{
  "matched": [
    {
      "TableId": "table_1",
      "Text": "extracted table text",
      "Left": 0.1,
      "Top": 0.2,
      "Width": 0.3,
      "Height": 0.4,
      "PageNo": 1,
      "unsan_line_no": 10,
      "unsan_text": "reference text",
      "bleu_score": 0.85
    }
  ],
  "unmatched": [...],
  "meta": {
    "file_name": "image.jpg"
  }
}
```

Directory Structure

— get_unmatched_tables_hybrid.py	# Main entry point
— requirements.txt	# Package dependencies
— config.json	# AWS configuration
— models/	# YOLO model directory
— images/	# Input images
— output/	# Processed results
— dump/	# Temporary files